

Poradnik do laboratorium

TCP i epoll po stronie serwera

API `l4-common.h`, wielu klientów, przesyłanie wiadomości

Materiał pomocniczy do L4: Sieć

Maj 2026

Ten poradnik jest nastawiony praktycznie: pokazuje, jak zbudować serwer TCP obsługujący wielu klientów, jak używać funkcji pomocniczych z `l4-common.h`, jak działa `epoll`, jak czytać i wysyłać wiadomości oraz jak unikać typowych błędów z laboratoriów sieciowych. Przykłady są pisane pod serwer, ponieważ to on zwykle ma więcej stanu, więcej deskryptorów i więcej pułapek.

Spis treści

1	Cel laboratorium TCP i epoll	2
2	Pliki i rola <code>l4-common.h</code>	2
2.1	Makro <code>ERR</code>	2
2.2	Makro <code>TEMP_FAILURE_RETRY</code>	3
2.3	<code>sethandler</code> : ustawianie obsługi sygnałów	3
2.4	<code>make_tcp_socket</code>	4
2.5	<code>bind_tcp_socket</code> : gotowy socket serwera	4
2.6	<code>add_new_client</code> : accept opakowany w funkcję	5
2.7	<code>bulk_read</code> i <code>bulk_write</code>	5
3	Minimalny serwer TCP bez epoll	6
4	Dlaczego potrzebny jest epoll	6
5	Najważniejsze funkcje epoll	7
5.1	<code>epoll_create1</code>	7
5.2	<code>epoll_ctl</code>	7
5.3	<code>epoll_wait</code>	7
5.4	Flagi zdarzeń	7
6	Wersja podstawowa: wielu klientów przez epoll	8
7	TCP jako strumień: jak odbierać wiadomości	10
7.1	Bufor klienta	10
7.2	Dopisanie danych do bufora i wyciąganie linii	10
8	Przechowywanie wielu klientów	11
8.1	Prosty wariant: tablica indeksowana deskryptorem	11
8.2	Lepszy wariant: <code>event.data.ptr</code>	12

9 Komunikacja serwera z klientem	13
9.1 Wysłanie krótkiej wiadomości	13
9.2 Czy write() może wysłać tylko część danych?	13
9.3 Wariant z buforem wyjściowym i EPOLLOUT	13
10 Identyfikacja klienta i stan protokołu	14
10.1 Obsługa pierwszej wiadomości klienta	15
10.2 Obsługa głosu po identyfikacji	15
11 Rozłączanie klientów	16
11.1 Jak rozpoznać rozłączenie	16
11.2 Co trzeba zrobić przy rozłączeniu	16
12 Czytanie danych w pętli zdarzeń	16
13 Pełniejszy szkielet serwera wyborczego	17
14 Jak testować serwer	20
14.1 Test pojedynczego klienta	20
14.2 Test wielu klientów	21
14.3 Test automatyczny prostymi komendami	21
15 Typowe błędy i jak je naprawić	21
15.1 Błąd: read() zwraca 0, a kod dalej przetwarza stary bajt	21
15.2 Błąd: zabijanie serwera przez ERR("read") przy EAGAIN	22
15.3 Błąd: głosy 0..2 zamiast 1..3 w protokole	22
15.4 Błąd: wysyłanie sizeof(MSG) zamiast strlen(MSG)	22
15.5 Błąd: brak czyszczenia stanu przy rozłączeniu	22
15.6 Błąd: pthread_create sprawdzany przez < 0	23
16 Wątki i dane współdzielone	23
17 Bezpieczne kończenie serwera	23
18 Jak policzyć wynik wyborów	24
19 Kompilacja	24
20 Krótka ściągą: co robić w którym miejscu	25
21 Checklist przed oddaniem	25
22 Podsumowanie	25

1 Cel laboratorium TCP i epoll

Na laboratorium chodzi o napisanie serwera, który:

- otwiera port TCP i nasłuchuje na połączenia,
- przyjmuje wielu klientów bez tworzenia osobnego procesu dla każdego klienta,
- reaguje na dane od klientów wtedy, gdy dane rzeczywiście są dostępne,
- poprawnie obsługuje rozłączenia,
- potrafi wysłać klientowi odpowiedź,
- przechowuje stan każdego klienta, np. identyfikator elektora, ostatni głos, bufor wejściowy,
- kończy pracę i zwalnia zasoby po sygnale.

Najważniejsza idea: serwer nie powinien blokować się na jednym kliencie. Jeżeli jeden klient połączy się i nic nie wyśle, inni klienci nadal muszą móc się łączyć i wysyłać wiadomości. Do tego służy epoll: jądro mówi programowi, które deskryptory są gotowe do operacji read, write albo accept.

Zapamiętaj: TCP to strumień bajtów, a nie strumień wiadomości. Jedno write() po stronie klienta nie musi odpowiadać jednemu read() po stronie serwera. Serwer musi mieć własny protokół, np. wiadomość kończy się znakiem '\n' albo ma stałą długość.

2 Pliki i rola l4-common.h

Plik l4-common.h jest pomocniczą biblioteką nagłówkową. Zawiera gotowe funkcje do:

- tworzenia socketów TCP i UNIX,
- bindowania socketu serwera do portu,
- łączenia klienta TCP z serwerem,
- przyjmowania klienta przez accept(),
- bezpieczniejszego instalowania handlerów sygnałów,
- powtarzania przerwanych wywołań systemowych,
- zapisywania i czytania dokładnej liczby bajtów w prostych protokołach.

W laboratorium najczęściej użyjesz tylko części TCP:

Listing 1: Najczęściej używane funkcje z l4-common.h przy serwerze TCP

```
1 int bind_tcp_socket(uint16_t port, int backlog_size);
2 int add_new_client(int sfd);
3 ssize_t bulk_read(int fd, char *buf, size_t count);
4 ssize_t bulk_write(int fd, char *buf, size_t count);
5 int sethandler(void (*f)(int), int sigNo);
```

Do testów klienta lub do kodu klienta TCP przydatne są też:

Listing 2: Funkcje przydatne po stronie klienta TCP

```
1 int connect_tcp_socket(char *name, char *port);
2 struct sockaddr_in make_address(char *address, char *port);
```

2.1 Makro ERR

Listing 3: Makro ERR z l4-common.h

```
1 #define ERR(source) \
2     (perror(source), fprintf(stderr, "%s:%d\n", __FILE__, __LINE__), exit(EXIT_FAILURE))
```

ERR("bind") wypisuje błąd z errno, plik i linię, a potem kończy program. To jest wygodne w kodzie laboratoryjnym, ale trzeba pamiętać: ERR nie nadaje się do sytuacji, które są normalne, np. klient się rozłączył albo read() na nieblokującym sockecie zwróciło EAGAIN. W takich przypadkach trzeba obsłużyć zdarzenie, a nie zabijać cały serwer.

2.2 Makro TEMP_FAILURE_RETRY

Listing 4: Makro ponawiające wywołanie przerwane sygnałem

```

1 #ifndef TEMP_FAILURE_RETRY
2 #define TEMP_FAILURE_RETRY(expression)      \
3     (__extension__({                        \
4         long int __result;                  \
5         do                                 \
6             __result = (long int)(expression); \
7             while (__result == -1L && errno == EINTR); \
8             __result;                        \
9     })))
10 #endif

```

Niektóre wywołania systemowe mogą zostać przerwane sygnałem i zwrócić -1 z errno == EINTR. Makro wykonuje wyrażenie ponownie, dopóki błąd wynika tylko z przzerwania sygnałem. Jest użyteczne przy read, write, accept, ale nie zastępuje normalnej obsługi EAGAIN, EWOULDBLOCK, ECONNRESET itd.

2.3 sethandler: ustawianie obsługi sygnałów

Listing 5: sethandler z l4-common.h

```

1 int sethandler(void (*f)(int), int sigNo)
2 {
3     struct sigaction act;
4     memset(&act, 0, sizeof(struct sigaction));
5     act.sa_handler = f;
6     if (-1 == sigaction(sigNo, &act, NULL))
7         return -1;
8     return 0;
9 }

```

Przykład użycia:

Listing 6: Minimalna obsługa SIGINT

```

1 volatile sig_atomic_t do_work = 1;
2
3 void sigint_handler(int sig)
4 {
5     (void)sig;
6     do_work = 0;
7 }
8
9 int main(void)
10 {
11     if (sethandler(sigint_handler, SIGINT) < 0)
12         ERR("sethandler");
13
14     while (do_work) {
15         /* petla serwera */
16     }
17 }

```

W handlerze sygnału nie wolno robić wszystkiego. Bezpieczny wzorzec to ustawienie flagi typu volatile sig_atomic_t, a właściwe sprzątnięcie wykonać później w głównej pętli.

2.4 make_tcp_socket

Listing 7: Tworzenie socketu TCP/IPv4

```

1 int make_tcp_socket(void)
2 {
3     int sock;
4     sock = socket(PF_INET, SOCK_STREAM, 0);
5     if (sock < 0)
6         ERR("socket");
7     return sock;
8 }

```

PF_INET oznacza IPv4, SOCK_STREAM oznacza TCP. To jeszcze nie jest serwer. To tylko utworzenie deskryptora socketu.

2.5 bind_tcp_socket: gotowy socket serwera

Listing 8: `bind_tcp_socket.c` – `common.h`

```

1 int bind_tcp_socket(uint16_t port, int backlog_size)
2 {
3     struct sockaddr_in addr;
4     int sockfd, t = 1;
5     sockfd = make_tcp_socket();
6     memset(&addr, 0, sizeof(struct sockaddr_in));
7     addr.sin_family = AF_INET;
8     addr.sin_port = htons(port);
9     addr.sin_addr.s_addr = htonl(INADDR_ANY);
10    if (setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, &t, sizeof(t)))
11        ERR("setsockopt");
12    if (bind(sockfd, (struct sockaddr *)&addr, sizeof(addr)) < 0)
13        ERR("bind");
14    if (listen(sockfd, backlog_size) < 0)
15        ERR("listen");
16    return sockfd;
17 }

```

Ta funkcja wykonuje trzy kroki serwera TCP:

1. `socket()` - tworzy socket,
2. `bind()` - przypina go do portu,
3. `listen()` - przełącza w tryb nasłuchiwanie.

SO_REUSEADDR pozwala szybko ponownie uruchomić serwer na tym samym porcie, nawet gdy poprzednie połączenia są jeszcze przez chwilę w stanie systemowym typu TIME_WAIT.

Przykład:

Listing 9:]Start serwera TCP na porcie z `argv[1]`

```

1 int main(int argc, char **argv)
2 {
3     if (argc != 2) {
4         fprintf(stderr, "Usage: %s port\n", argv[0]);
5         return EXIT_FAILURE;
6     }
7
8     uint16_t port = (uint16_t)atoi(argv[1]);
9     int sfd = bind_tcp_socket(port, 10);
10
11    printf("Server listens on port %u\n", port);
12
13    close(sfd);
14    return EXIT_SUCCESS;
15 }

```

2.6 add_new_client: accept opakowany w funkcję

Listing 10: Przyjmowanie nowego klienta

```
1 int add_new_client(int sfd)
2 {
3     int nfd;
4     if ((nfd = TEMP_FAILURE_RETRY(accept(sfd, NULL, NULL))) < 0)
5     {
6         if (EAGAIN == errno || EWOULDBLOCK == errno)
7             return -1;
8         ERR("accept");
9     }
10    return nfd;
11 }
```

sfd to socket nasłuchujący. Funkcja zwraca nowy deskryptor nfd, który reprezentuje konkretną rozmowę z konkretnym klientem. Od tej chwili:

- sfd służy dalej do przyjmowania następnych klientów,
- nfd służy do read() i write() z tym jednym klientem.

Częsty błąd: nie czytaj danych klienta z sfd. Z sfd robisz tylko accept(). Dane konkretnego klienta czytasz z deskryptora zwróconego przez accept(), czyli z nfd.

2.7 bulk_read i bulk_write

Listing 11: Czytanie dokładnie count bajtów albo do EOF

```
1 ssize_t bulk_read(int fd, char *buf, size_t count)
2 {
3     int c;
4     size_t len = 0;
5     do
6     {
7         c = TEMP_FAILURE_RETRY(read(fd, buf, count));
8         if (c < 0)
9             return c;
10        if (0 == c)
11            return len;
12        buf += c;
13        len += c;
14        count -= c;
15    } while (count > 0);
16    return len;
17 }
```

Listing 12: Zapisywanie dokładnie count bajtów

```
1 ssize_t bulk_write(int fd, char *buf, size_t count)
2 {
3     int c;
4     size_t len = 0;
5     do
6     {
7         c = TEMP_FAILURE_RETRY(write(fd, buf, count));
8         if (c < 0)
9             return c;
10        buf += c;
11        len += c;
12        count -= c;
13    } while (count > 0);
14    return len;
15 }
```

Te funkcje są dobre, gdy protokół ma stałą długość komunikatu, np. zawsze czytasz 4 bajty liczby albo zawsze wysyłasz pełny napis powitalny. Są mniej wygodne w protokole tekstowym, gdzie klient wpisuje linie różnej długości.

Uwaga przy `epoll` i nieblokujących socketach: `bulk_read` może zwrócić `-1` z `errno == EAGAIN`, gdy w danej chwili nie ma pełnej liczby bajtów. W serwerze zdarzeniowym częściej używa się zwykłego `read()` do lokalnego bufora, a potem parsuje to, co przyszło.

3 Minimalny serwer TCP bez epoll

Zanim przejdziemy do wielu klientów, warto rozumieć wersję najprostszą.

Listing 13: Serwer przyjmujący jednego klienta i wysyłający wiadomość

```
1 #include "l4-common.h"
2
3 int main(int argc, char **argv)
4 {
5     if (argc != 2) {
6         fprintf(stderr, "Usage: %s port\n", argv[0]);
7         return EXIT_FAILURE;
8     }
9
10    int sfd = bind_tcp_socket((uint16_t)atoi(argv[1]), 10);
11    int cfd = add_new_client(sfd);
12
13    if (cfd >= 0) {
14        char msg[] = "Welcome from server\n";
15        if (bulk_write(cfd, msg, strlen(msg)) < 0)
16            ERR("bulk_write");
17        close(cfd);
18    }
19
20    close(sfd);
21    return EXIT_SUCCESS;
22 }
```

Test:

Listing 14: Uruchomienie i test przez netcat

```
1 # terminal 1
2 ./server 8888
3
4 # terminal 2
5 nc localhost 8888
```

Ten serwer jest łatwy, ale nie rozwiązuje problemu wielu klientów. Jeżeli użyjesz zwykłego `read(cfd, ...)` w pętli dla jednego klienta, program może czekać na niego bez końca. W tym czasie inni klienci nie zostaną obsłużeni.

4 Dlaczego potrzebny jest epoll

`epoll` rozwiązuje problem: „mam wiele deskryptorów, chcę spać, dopóki któryś będzie gotowy”. Deskryptorami mogą być:

- socket nasłuchujący serwera - gotowy oznacza, że można wykonać `accept()`,
- socket klienta - gotowy oznacza, że można wykonać `read()`,
- czasem socket klienta gotowy do pisania - można wykonać część `write()`.

Model pracy:

1. utwórz socket serwera przez `bind_tcp_socket`,
2. utwórz instancję `epoll` przez `epoll_create1(0)`,

3. dodaj socket serwera do epoll,
4. wejdź w pętlę `epoll_wait`,
5. dla każdego zdarzenia sprawdź, którego deskryptora dotyczy,
6. jeżeli to socket serwera - przyjmij nowego klienta,
7. jeżeli to socket klienta - czytaj dane albo obsłuż rozłączenie.

5 Najważniejsze funkcje epoll

5.1 `epoll_create1`

Listing 15: Tworzenie obiektu epoll

```
1 int efd = epoll_create1(0);
2 if (efd < 0)
3     ERR("epoll_create1");
```

`efd` to deskryptor obiektu epoll. Nie jest to socket. To obiekt jądra, do którego dodajesz inne deskryptory.

5.2 `epoll_ctl`

Listing 16: Dodanie deskryptora do epoll

```
1 struct epoll_event ev;
2 ev.events = EPOLLIN;
3 ev.data.fd = sfd;
4
5 if (epoll_ctl(efd, EPOLL_CTL_ADD, sfd, &ev) < 0)
6     ERR("epoll_ctl ADD sfd");
```

Najczęstsze operacje:

- `EPOLL_CTL_ADD` - dodaj deskryptor,
- `EPOLL_CTL_MOD` - zmień obserwowane zdarzenia,
- `EPOLL_CTL_DEL` - usuń deskryptor.

5.3 `epoll_wait`

Listing 17: Czekanie na zdarzenia

```
1 #define MAX_EVENTS 32
2 struct epoll_event events[MAX_EVENTS];
3
4 int n = epoll_wait(efd, events, MAX_EVENTS, -1);
5 if (n < 0) {
6     if (errno == EINTR) {
7         /* signal interrupted waiting; check do_work and continue/stop */
8     } else {
9         ERR("epoll_wait");
10    }
11 }
```

Ostatni argument to timeout w milisekundach. `-1` oznacza czekanie bez limitu. `0` oznacza natychmiastowy powrót.

5.4 Flagi zdarzeń

Flaga	Znaczenie
EPOLLIN	Można czytać: na sockecie klienta są dane, a na sockecie nasłuchującym czeka połączenie do <code>accept()</code> .
EPOLLOUT	Można pisać bez natychmiastowego zablokowania. Używane przy większych buforach wyjściowych.
EPLLERR	Wystąpił błąd na deskrytorze. Zwykle trzeba zamknąć klienta.
EPOLLHUP	Druga strona zamknęła połączenie lub deskryptor jest w stanie hang-up.
EPOLLRDHUP	Druga strona zamknęła pół połączenia do pisania; bardzo przydatne do wykrywania rozłączenia klienta TCP. Wymaga <code>_GNU_SOURCE</code> .
EPOLLET	Edge-triggered mode. Bardziej wydajne, ale trudniejsze; trzeba czytać w pętli do <code>EAGAIN</code> . Na laboratorium zwykle wystarczy tryb domyślny, czyli level-triggered.

Na laboratorium najprostszy bezpieczny zestaw dla klienta to często `EPOLLIN` | `EPOLLRDHUP`. Przy zdarzeniu z `EPLLERR`, `EPOLLHUP` albo `EPOLLRDHUP` zamknij połączenie.

6 Wersja podstawowa: wielu klientów przez epoll

Poniższy przykład pokazuje rdzeń serwera: przyjmowanie wielu klientów, wysyłanie powitania i wypisywanie danych od klientów. To jest bardzo bliskie temu, czego zwykle wymaga etap „wielu klientów”.

Listing 18: Szkielet serwera TCP z epoll - wielu klientów

```

1 #include "l4-common.h"
2
3 #define MAX_EVENTS 32
4 #define BACKLOG 16
5
6 static void set_nonblock(int fd)
7 {
8     int flags = fcntl(fd, F_GETFL, 0);
9     if (flags < 0)
10         ERR("fcntl F_GETFL");
11     if (fcntl(fd, F_SETFL, flags | O_NONBLOCK) < 0)
12         ERR("fcntl F_SETFL");
13 }
14
15 static void add_fd_to_epoll(int efd, int fd, uint32_t events)
16 {
17     struct epoll_event ev;
18     memset(&ev, 0, sizeof(ev));
19     ev.events = events;
20     ev.data.fd = fd;
21     if (epoll_ctl(efd, EPOLL_CTL_ADD, fd, &ev) < 0)
22         ERR("epoll_ctl ADD");
23 }
24
25 static void close_client(int efd, int fd)
26 {
27     epoll_ctl(efd, EPOLL_CTL_DEL, fd, NULL);
28     close(fd);
29 }
30

```

```

31 int main(int argc, char **argv)
32 {
33     if (argc != 2) {
34         fprintf(stderr, "Usage: %s port\n", argv[0]);
35         return EXIT_FAILURE;
36     }
37
38     int sfd = bind_tcp_socket((uint16_t)atoi(argv[1]), BACKLOG);
39     set_nonblock(sfd);
40
41     int efd = epoll_create1(0);
42     if (efd < 0)
43         ERR("epoll_create1");
44
45     add_fd_to_epoll(efd, sfd, EPOLLIN);
46
47     struct epoll_event events[MAX_EVENTS];
48
49     for (;;) {
50         int n = epoll_wait(efd, events, MAX_EVENTS, -1);
51         if (n < 0) {
52             if (errno == EINTR)
53                 continue;
54             ERR("epoll_wait");
55         }
56
57         for (int i = 0; i < n; ++i) {
58             int fd = events[i].data.fd;
59             uint32_t revents = events[i].events;
60
61             if (fd == sfd) {
62                 for (;;) {
63                     int cfd = add_new_client(sfd);
64                     if (cfd < 0)
65                         break; /* no more clients waiting now */
66
67                     set_nonblock(cfd);
68                     add_fd_to_epoll(efd, cfd, EPOLLIN | EPOLLRDHUP);
69
70                     char hello[] = "Welcome, client!\n";
71                     if (bulk_write(cfd, hello, strlen(hello)) < 0)
72                         close_client(efd, cfd);
73                 }
74             } else {
75                 if (revents & (EPOLLERR | EPOLLHUP | EPOLLRDHUP)) {
76                     close_client(efd, fd);
77                     continue;
78                 }
79
80                 if (revents & EPOLLIN) {
81                     char buf[256];
82                     ssize_t r = read(fd, buf, sizeof(buf) - 1);
83
84                     if (r > 0) {
85                         buf[r] = '\0';
86                         printf("fd %d says: %s", fd, buf);
87                     } else if (r == 0) {
88                         close_client(efd, fd);
89                     } else if (errno != EAGAIN && errno != EWOULDBLOCK) {
90                         close_client(efd, fd);
91                     }
92                 }
93             }
94         }
95     }
96 }

```

Co tu się dzieje:

- socket serwera jest nieblokujący, więc można bezpiecznie akceptować w pętli aż do `EAGAIN`,
- każdy klient po `accept()` też jest ustawiany jako nieblokujący,
- klient jest dodawany do `epoll`, żeby serwer dostał zdarzenie, gdy klient coś wyśle,
- `fd == sfd` oznacza nowe połączenie,
- `fd != sfd` oznacza zdarzenie na istniejącym kliencie,
- `read() == 0` oznacza normalne zamknięcie połączenia przez klienta,
- przy rozłączeniu usuwamy deskryptor z `epoll` i zamykamy `close(fd)`.

7 TCP jako strumień: jak odbierać wiadomości

Wiele osób zakłada, że jeżeli klient wyśle "abc\n", to serwer w jednym `read()` dostanie dokładnie "abc\n". Tak nie musi być.

Możliwe sytuacje:

- klient wysła "abc\n", serwer czyta "a", potem "bc\n",
- klient wysła dwie linie, serwer czyta obie naraz,
- serwer czyta pół komendy i musi poczekać na resztę.

Dlatego protokół powinien mieć granice wiadomości. Najprostsza granica to znak nowej linii.

7.1 Bufor klienta

Każdy klient powinien mieć własny bufor wejściowy. Nie wystarczy jedna globalna tablica `char buf[256]`, jeśli chcesz składać częściowe wiadomości.

Listing 19: Stan jednego klienta z buforem wejściowym

```
1 #define INBUF_SIZE 1024
2
3 typedef struct client_state {
4     int fd;
5     int identified;
6     int elector_id;
7     char inbuf[INBUF_SIZE];
8     size_t inbuf_len;
9 } client_state;
```

7.2 Dopisanie danych do bufora i wyciąganie linii

Listing 20: Przetwarzanie wiadomości zakończonych znakiem nowej linii

```
1 static void handle_line(client_state *c, const char *line)
2 {
3     printf("client %d line: %s\n", c->fd, line);
4 }
5
6 static void consume_input(client_state *c, const char *data, size_t len)
7 {
8     if (len > INBUF_SIZE - c->inbuf_len)
9         len = INBUF_SIZE - c->inbuf_len; /* or disconnect: input too long */
10
11     memcpy(c->inbuf + c->inbuf_len, data, len);
12     c->inbuf_len += len;
13
14     size_t start = 0;
```

```

15 for (size_t i = 0; i < c->inbuf_len; ++i) {
16     if (c->inbuf[i] == '\n') {
17         c->inbuf[i] = '\0';
18
19         if (i > start && c->inbuf[i - 1] == '\r')
20             c->inbuf[i - 1] = '\0';
21
22         handle_line(c, c->inbuf + start);
23         start = i + 1;
24     }
25 }
26
27 if (start > 0) {
28     memmove(c->inbuf, c->inbuf + start, c->inbuf_len - start);
29     c->inbuf_len -= start;
30 }
31 }

```

To jest wzorzec dobry do komend tekstowych typu:

Listing 21: Przykładowy protokół tekstowy

```

1 ID 1
2 VOTE 2
3 VOTE 3
4 QUIT

```

Na prostym etapie, gdzie klient wysyła pojedynczy znak, można czytać po 1 bajcie. Ale na prawdziwym serwerze laboratoryjnym umiejętność buforowania linii jest bardzo przydatna.

8 Przechowywanie wielu klientów

8.1 Prosty wariant: tablica indeksowana deskryptorem

Deskryptory są liczbami całkowitymi. Można stworzyć tablicę `clients[FD_LIMIT]` i używać `clients[fd]`. To jest proste, ale trzeba znać rozsądny limit.

Listing 22: Tablica klientów indeksowana fd

```

1 #define FD_LIMIT 1024
2
3 typedef struct client_state {
4     int active;
5     int fd;
6     int identified;
7     int elector_id;
8 } client_state;
9
10 client_state clients[FD_LIMIT];
11
12 static client_state *get_client(int fd)
13 {
14     if (fd < 0 || fd >= FD_LIMIT)
15         return NULL;
16     return clients[fd].active ? &clients[fd] : NULL;
17 }

```

Dodanie klienta:

Listing 23: Rejestracja nowego klienta w tablicy

```

1 static void register_client(int fd)
2 {
3     if (fd >= FD_LIMIT) {
4         close(fd);

```

```

5     return;
6 }
7
8 clients[fd].active = 1;
9 clients[fd].fd = fd;
10 clients[fd].identified = 0;
11 clients[fd].elector_id = -1;
12 }

```

Usunięcie klienta:

Listing 24: Usuwanie klienta z tablicy i z epoll

```

1 static void unregister_client(int efd, int fd)
2 {
3     if (fd >= 0 && fd < FD_LIMIT)
4         memset(&clients[fd], 0, sizeof(clients[fd]));
5
6     epoll_ctl(efd, EPOLL_CTL_DEL, fd, NULL);
7     close(fd);
8 }

```

8.2 Lepszy wariant: event.data.ptr

epoll_event ma pole data, w którym można przechować fd, ale też wskaźnik. Dzięki temu zdarzenie od razu niesie wskaźnik na stan klienta.

Listing 25: Użycie data.ptr dla klienta

```

1 typedef enum {
2     CONN_LISTENER,
3     CONN_CLIENT
4 } conn_type;
5
6 typedef struct connection {
7     conn_type type;
8     int fd;
9     int identified;
10    int elector_id;
11 } connection;
12
13 static void add_connection(int efd, connection *c, uint32_t events)
14 {
15     struct epoll_event ev;
16     memset(&ev, 0, sizeof(ev));
17     ev.events = events;
18     ev.data.ptr = c;
19
20     if (epoll_ctl(efd, EPOLL_CTL_ADD, c->fd, &ev) < 0)
21         ERR("epoll_ctl ADD");
22 }

```

W pętli:

Listing 26: Odbiór zdarzenia przez data.ptr

```

1 connection *c = events[i].data.ptr;
2
3 if (c->type == CONN_LISTENER) {
4     /* accept new clients */
5 } else if (c->type == CONN_CLIENT) {
6     /* read from c->fd */
7 }

```

To jest bardziej skalowalne, ale wymaga zarządzania pamięcią: klient po rozłączeniu musi mieć zwolnioną strukturę, np. free(c) po epoll_ctl DEL i close.

9 Komunikacja serwera z klientem

9.1 Wysłanie krótkiej wiadomości

Najprościej:

Listing 27: Wysłanie krótkiego komunikatu

```
1 static int send_text(int fd, const char *text)
2 {
3     ssize_t n = bulk_write(fd, (char *)text, strlen(text));
4     return n < 0 ? -1 : 0;
5 }
```

Użycie:

Listing 28: Powitanie klienta

```
1 if (send_text(cfd, "Welcome, elector!\n") < 0) {
2     close(cfd);
3 }
```

Warto używać `strlen`, a nie `sizeof` na makrze tekstowym, jeżeli nie chcesz wysłać bajtu `'\0'`.

Listing 29: Różnica `sizeof` i `strlen`

```
1 #define MSG "Hello\n"
2
3 sizeof(MSG) /* 7: H e l l o \n \0 */
4 strlen(MSG) /* 6: H e l l o \n */
```

9.2 Czy `write()` może wysłać tylko część danych?

Tak. Dla socketu TCP `write(fd, buf, len)` może zwrócić wartość dodatnią mniejszą niż `len`. Dlatego `bulk_write` ponawia zapis, aż wyśle wszystko albo wystąpi błąd.

Dla krótkich komunikatów na blokującym sockecie zwykle działa to bezproblemowo. Dla nieblokującego socketu trzeba pamiętać o `EAGAIN`. W pełnym serwerze produkcyjnym robi się bufor wyjściowy per klient i włącza `EPOLLOUT`. Na laboratorium często wystarczy krótkie `bulk_write`, ale trzeba wiedzieć, że to uproszczenie.

9.3 Wariant z buforem wyjściowym i `EPOLLOUT`

Gdy komunikaty są większe albo socket jest nieblokujący, dobry wzorzec to:

1. dopisz odpowiedź do bufora klienta,
2. zmień obserwowane zdarzenia na `EPOLLIN | EPOLLOUT | EPOLLRDHUP`,
3. gdy przyjdzie `EPOLLOUT`, wyślij tyle, ile się da,
4. jeśli bufor pusty, wyłącz `EPOLLOUT`.

Minimalny zarys:

Listing 30: Bufor wyjściowy klienta - zarys

```
1 #define OUTBUF_SIZE 2048
2
3 typedef struct client_state {
4     int fd;
5     char outbuf[OUTBUF_SIZE];
6     size_t out_len;
7     size_t out_sent;
8 } client_state;
9
10 static void want_write(int efd, client_state *c)
```

```

11 {
12     struct epoll_event ev;
13     memset(&ev, 0, sizeof(ev));
14     ev.events = EPOLLIN | EPOLLOUT | EPOLLRDHUP;
15     ev.data.fd = c->fd;
16     if (epoll_ctl(efd, EPOLL_CTL_MOD, c->fd, &ev) < 0)
17         ERR("epoll_ctl MOD EPOLLOUT");
18 }
19
20 static void stop_write(int efd, client_state *c)
21 {
22     struct epoll_event ev;
23     memset(&ev, 0, sizeof(ev));
24     ev.events = EPOLLIN | EPOLLRDHUP;
25     ev.data.fd = c->fd;
26     if (epoll_ctl(efd, EPOLL_CTL_MOD, c->fd, &ev) < 0)
27         ERR("epoll_ctl MOD no EPOLLOUT");
28 }
29
30 static int flush_output(int efd, client_state *c)
31 {
32     while (c->out_sent < c->out_len) {
33         ssize_t n = write(c->fd,
34                         c->outbuf + c->out_sent,
35                         c->out_len - c->out_sent);
36         if (n > 0) {
37             c->out_sent += (size_t)n;
38         } else if (n < 0 && (errno == EAGAIN || errno == EWOULDBLOCK)) {
39             return 0;
40         } else {
41             return -1;
42         }
43     }
44
45     c->out_len = 0;
46     c->out_sent = 0;
47     stop_write(efd, c);
48     return 0;
49 }

```

Na laboratorium nie zawsze trzeba implementować pełny bufor wyjściowy, ale warto rozumieć, dlaczego EPOLLOUT istnieje.

10 Identyfikacja klienta i stan protokołu

W zadaniu typu „elektorzy” klient najpierw podaje numer elektora, a dopiero potem może wysłać głosy. To znaczy, że klient ma stan.

Listing 31: Stan klienta w prostym protokole wyborów

```

1 typedef enum {
2     CLIENT_WAITING_FOR_ID,
3     CLIENT_IDENTIFIED
4 } client_phase;
5
6 typedef struct client_state {
7     int active;
8     int fd;
9     client_phase phase;
10    int elector_id; /* 0..6 or -1 */
11 } client_state;

```

Dodatkowo serwer ma stan globalny:

Listing 32: Stan globalny wyborów

```

1 #define ELECTORS_NUM 7
2 #define CANDIDATES_NUM 3
3
4 int elector_fd[ELECTORS_NUM]; /* -1 when elector is offline */
5 int votes[ELECTORS_NUM];      /* -1 no vote, otherwise 0..2 */

```

Inicjalizacja:

Listing 33: Inicjalizacja stanu wyborów

```

1 for (int i = 0; i < ELECTORS_NUM; ++i) {
2     elector_fd[i] = -1;
3     votes[i] = -1;
4 }

```

10.1 Obsługa pierwszej wiadomości klienta

Listing 34: Identyfikacja elektora

```

1 static const char *elector_names[ELECTORS_NUM] = {
2     "Mainz",
3     "Trier",
4     "Cologne",
5     "Bohemia",
6     "Palatinate",
7     "Saxony",
8     "Brandenburg"
9 };
10
11 static int identify_client(client_state *c, char ch, int elector_fd[ELECTORS_NUM])
12 {
13     if (ch < '1' || ch > '7')
14         return -1; /* invalid id */
15
16     int id = ch - '1';
17
18     if (elector_fd[id] != -1)
19         return -1; /* this elector is already connected */
20
21     elector_fd[id] = c->fd;
22     c->elector_id = id;
23     c->phase = CLIENT_IDENTIFIED;
24
25     char msg[128];
26     snprintf(msg, sizeof(msg), "Welcome, elector of %s!\n", elector_names[id]);
27     if (bulk_write(c->fd, msg, strlen(msg)) < 0)
28         return -1;
29
30     return 0;
31 }

```

Najważniejsze reguły:

- przed identyfikacją klient może wysłać tylko cyfrę '1'..'7',
- jeśli wyśle coś innego, zamykasz połączenie,
- jeśli ten elektor już jest połączony, zamykasz nowe połączenie,
- po poprawnej identyfikacji wysyłasz wiadomość powitalną dopasowaną do elektora.

10.2 Obsługa głosu po identyfikacji

Listing 35: Przyjmowanie głosu 1..3

```

1 static void handle_vote(client_state *c, char ch, int votes[ELECTORS_NUM])
2 {

```

```
3  if (ch < '1' || ch > '3')
4      return; /* ignore other characters */
5
6  int candidate = ch - '1';
7  votes[c->elector_id] = candidate;
8
9  printf("Elector %d voted for candidate %d\n",
10         c->elector_id + 1,
11         candidate + 1);
12 }
```

Zwróć uwagę na numerację:

- w protokole klient wysyła '1', '2' albo '3',
- w tablicy wygodnie trzymać 0, 1, 2,
- przy wypisywaniu dla człowieka zwykle dodajesz +1.

11 Rozłączanie klientów

Rozłączenie klienta to nie awaria całego serwera. To normalna sytuacja.

11.1 Jak rozpoznać rozłączenie

Najczęstsze sygnały:

- `read(fd, ...)` zwraca 0,
- `events[i].events` ma `EPOLLRDHUP`, `EPOLLHUP` albo `EPOLLERR`,
- `read()` zwraca -1 z błędem typu `ECONNRESET`.

11.2 Co trzeba zrobić przy rozłączeniu

Listing 36: Sprzątanie po kliencie-elektorze

```
1 static void disconnect_client(int efd,
2                               client_state *c,
3                               int elector_fd[ELECTORS_NUM])
4 {
5     if (c->phase == CLIENT_IDENTIFIED && c->elector_id >= 0) {
6         elector_fd[c->elector_id] = -1;
7     }
8
9     epoll_ctl(efd, EPOLL_CTL_DEL, c->fd, NULL);
10    close(c->fd);
11
12    c->active = 0;
13    c->fd = -1;
14    c->phase = CLIENT_WAITING_FOR_ID;
15    c->elector_id = -1;
16 }
```

W zadaniu wyborczym ważne jest, żeby po rozłączeniu elektora pozwolić mu połączyć się ponownie. Dlatego trzeba wyczyścić informację „ten elektor jest online”. Głosu zwykle nie kasujesz, bo ostatni oddany głos nadal jest jego głosem, chyba że treść zadania mówi inaczej.

12 Czytanie danych w pętli zdarzeń

W trybie nieblokującym typowy handler danych od klienta wygląda tak:

Listing 37: Czytanie danych od klienta w serwerze epoll

```

1 static int read_from_client(client_state *c)
2 {
3     for (;;) {
4         char buf[256];
5         ssize_t n = read(c->fd, buf, sizeof(buf));
6
7         if (n > 0) {
8             for (ssize_t i = 0; i < n; ++i) {
9                 char ch = buf[i];
10                if (c->phase == CLIENT_WAITING_FOR_ID) {
11                    /* call identify_client(...) */
12                } else {
13                    /* call handle_vote(...) */
14                }
15            }
16        } else if (n == 0) {
17            return 0; /* disconnected */
18        } else {
19            if (errno == EAGAIN || errno == EWOULDBLOCK)
20                return 1; /* no more data now, client still alive */
21            if (errno == EINTR)
22                continue;
23            return 0; /* real error, disconnect */
24        }
25    }
26 }

```

Jeżeli nie używasz EPOLLET, nie musisz zawsze czytać w pętli do EAGAIN, ale jest to dobry nawyk przy nieblokujących socketach. Przy EPOLLET jest to konieczne.

13 Pełniejszy szkielet serwera wyborczego

Ten przykład łączy najważniejsze elementy: `l4-common.h`, `epoll`, wielu klientów, identyfikację, głosowanie i rozłączanie. To nie jest jedyny możliwy styl, ale dobrze pokazuje architekturę serwera.

Listing 38: Szkielet serwera: wielu klientów, identyfikacja i głosy

```

1 #include "l4-common.h"
2
3 #define MAX_EVENTS 32
4 #define FD_LIMIT 1024
5 #define ELECTORS_NUM 7
6
7 volatile sig_atomic_t do_work = 1;
8
9 static const char *elector_names[ELECTORS_NUM] = {
10     "Mainz", "Trier", "Cologne", "Bohemia",
11     "Palatinate", "Saxony", "Brandenburg"
12 };
13
14 typedef enum {
15     WAITING_FOR_ID,
16     IDENTIFIED
17 } phase_t;
18
19 typedef struct client_state {
20     int active;
21     int fd;
22     phase_t phase;
23     int elector_id;
24 } client_state;
25
26 static client_state clients[FD_LIMIT];

```

```

27 static int elector_fd[ELECTORS_NUM];
28 static int votes[ELECTORS_NUM];
29
30 static void sigint_handler(int sig)
31 {
32     (void)sig;
33     do_work = 0;
34 }
35
36 static void set_nonblock(int fd)
37 {
38     int flags = fcntl(fd, F_GETFL, 0);
39     if (flags < 0)
40         ERR("fcntl F_GETFL");
41     if (fcntl(fd, F_SETFL, flags | O_NONBLOCK) < 0)
42         ERR("fcntl F_SETFL");
43 }
44
45 static void epoll_add(int efd, int fd, uint32_t events)
46 {
47     struct epoll_event ev;
48     memset(&ev, 0, sizeof(ev));
49     ev.events = events;
50     ev.data.fd = fd;
51     if (epoll_ctl(efd, EPOLL_CTL_ADD, fd, &ev) < 0)
52         ERR("epoll_ctl ADD");
53 }
54
55 static void disconnect_client(int efd, int fd)
56 {
57     if (fd < 0 || fd >= FD_LIMIT || !clients[fd].active)
58         return;
59
60     client_state *c = &clients[fd];
61
62     if (c->phase == IDENTIFIED && c->elector_id >= 0)
63         elector_fd[c->elector_id] = -1;
64
65     epoll_ctl(efd, EPOLL_CTL_DEL, fd, NULL);
66     close(fd);
67     memset(c, 0, sizeof(*c));
68     c->fd = -1;
69 }
70
71 static void send_welcome(int fd, int elector_id)
72 {
73     char msg[128];
74     snprintf(msg, sizeof(msg), "Welcome, elector of %s!\n",
75             elector_names[elector_id]);
76     bulk_write(fd, msg, strlen(msg));
77 }
78
79 static int identify_client(client_state *c, char ch)
80 {
81     if (ch < '1' || ch > '7')
82         return -1;
83
84     int id = ch - '1';
85     if (elector_fd[id] != -1)
86         return -1;
87
88     elector_fd[id] = c->fd;
89     c->phase = IDENTIFIED;
90     c->elector_id = id;
91     send_welcome(c->fd, id);
92     return 0;
93 }

```

```
94
95 static void handle_vote(client_state *c, char ch)
96 {
97     if (ch < '1' || ch > '3')
98         return;
99
100     votes[c->elector_id] = ch - '1';
101     printf("Elector %d voted for candidate %d\n",
102           c->elector_id + 1, votes[c->elector_id] + 1);
103 }
104
105 static int read_client_data(client_state *c)
106 {
107     for (;;) {
108         char buf[256];
109         ssize_t n = read(c->fd, buf, sizeof(buf));
110
111         if (n > 0) {
112             for (ssize_t i = 0; i < n; ++i) {
113                 if (c->phase == WAITING_FOR_ID) {
114                     if (identify_client(c, buf[i]) < 0)
115                         return 0;
116                     } else {
117                         handle_vote(c, buf[i]);
118                     }
119             }
120         } else if (n == 0) {
121             return 0;
122         } else {
123             if (errno == EAGAIN || errno == EWOULDBLOCK)
124                 return 1;
125             if (errno == EINTR)
126                 continue;
127             return 0;
128         }
129     }
130 }
131
132 int main(int argc, char **argv)
133 {
134     if (argc != 2) {
135         fprintf(stderr, "Usage: %s port\n", argv[0]);
136         return EXIT_FAILURE;
137     }
138
139     if (sethandler(sigint_handler, SIGINT) < 0)
140         ERR("sethandler");
141
142     for (int i = 0; i < FD_LIMIT; ++i)
143         clients[i].fd = -1;
144     for (int i = 0; i < ELECTORS_NUM; ++i) {
145         elector_fd[i] = -1;
146         votes[i] = -1;
147     }
148
149     int sfd = bind_tcp_socket((uint16_t)atoi(argv[1]), 16);
150     set_nonblock(sfd);
151
152     int efd = epoll_create1(0);
153     if (efd < 0)
154         ERR("epoll_create1");
155
156     epoll_add(efd, sfd, EPOLLIN);
157
158     struct epoll_event events[MAX_EVENTS];
159
160     while (do_work) {
```

```

161     int n = epoll_wait(efd, events, MAX_EVENTS, -1);
162     if (n < 0) {
163         if (errno == EINTR)
164             continue;
165         ERR("epoll_wait");
166     }
167
168     for (int i = 0; i < n; ++i) {
169         int fd = events[i].data.fd;
170         uint32_t revents = events[i].events;
171
172         if (fd == sfd) {
173             for (;;) {
174                 int cfd = add_new_client(sfd);
175                 if (cfd < 0)
176                     break;
177                 if (cfd >= FD_LIMIT) {
178                     close(cfd);
179                     continue;
180                 }
181
182                 set_nonblock(cfd);
183                 clients[cfd].active = 1;
184                 clients[cfd].fd = cfd;
185                 clients[cfd].phase = WAITING_FOR_ID;
186                 clients[cfd].elector_id = -1;
187                 epoll_add(efd, cfd, EPOLLIN | EPOLLRDHUP);
188             }
189         } else {
190             if (revents & (EPOLLERR | EPOLLHUP | EPOLLRDHUP)) {
191                 disconnect_client(efd, fd);
192                 continue;
193             }
194             if (revents & EPOLLIN) {
195                 if (!read_client_data(&clients[fd]))
196                     disconnect_client(efd, fd);
197             }
198         }
199     }
200 }
201
202 for (int fd = 0; fd < FD_LIMIT; ++fd) {
203     if (clients[fd].active)
204         disconnect_client(efd, fd);
205 }
206
207 close(efd);
208 close(sfd);
209 return EXIT_SUCCESS;
210 }

```

14 Jak testować serwer

14.1 Test pojedynczego klienta

Listing 39: Terminal 1: serwer

```
1 ./sop-hre 8888
```

Listing 40: Terminal 2: klient netcat

```
1 nc localhost 8888
```

Wpisz w kliencie:

Listing 41: Przykładowa rozmowa klienta

```
1 1
2 2
3 3
```

Jeżeli protokół czyta pojedyncze znaki, znak nowej linii zostanie odebrany jako dodatkowy znak. Dla głosowania powinien być ignorowany.

14.2 Test wielu klientów

Otwórz kilka terminali:

Listing 42: Kilku klientów naraz

```
1 # terminal 2
2 nc localhost 8888
3
4 # terminal 3
5 nc localhost 8888
6
7 # terminal 4
8 nc localhost 8888
```

Sprawdź przypadki:

- klient 1 identyfikuje się jako 1, klient 2 jako 2,
- klient 3 próbuje też zidentyfikować się jako 1 - powinien zostać rozłączony,
- klient 1 rozłącza się, potem nowy klient może wejść jako 1,
- klient wysyła znaki inne niż 1..3 po identyfikacji - serwer ignoruje je,
- klient wysyła błędny identyfikator przed identyfikacją - serwer zamyka połączenie.

14.3 Test automatyczny prostymi komendami

Listing 43: Wysłanie identyfikatora i głosu potokiem

```
1 printf '1\n2\n' | nc localhost 8888
2 printf '2\n3\n' | nc localhost 8888
```

Uwaga: taki klient szybko się rozłączy po wysłaniu danych. To dobry test obsługi EOF.

15 Typowe błędy i jak je naprawić

15.1 Błąd: `read()` zwraca 0, a kod dalej przetwarza stary bajt

Zły wzorzec:

Listing 44: Niepoprawne: po `n == 0` kod może iść dalej

```
1 ssize_t n = read(fd, &msg, 1);
2 if (n == 0) {
3     close(fd);
4 }
5 /* tutaj msg może być stary albo niezainicjalizowany */
```

Poprawny wzorzec:

Listing 45: Poprawne: po rozłączeniu przechodzimy dalej

```
1 ssize_t n = read(fd, &msg, 1);
2 if (n == 0) {
3     disconnect_client(efd, fd);
4     continue;
5 }
```

```
6 if (n < 0) {
7     if (errno == EAGAIN || errno == EWOULDBLOCK)
8         continue;
9     disconnect_client(efd, fd);
10    continue;
11 }
12 /* tutaj n > 0, msg jest poprawny */
```

15.2 Błąd: zabijanie serwera przez ERR("read") przy EAGAIN

Na nieblokującym sockecie EAGAIN oznacza: „teraz nie ma danych”. To nie jest błąd krytyczny. Nie używaj wtedy ERR.

Listing 46: Obsługa EAGAIN

```
1 ssize_t n = read(fd, buf, sizeof(buf));
2 if (n < 0) {
3     if (errno == EAGAIN || errno == EWOULDBLOCK)
4         return; /* all data consumed */
5     disconnect_client(efd, fd);
6 }
```

15.3 Błąd: głosy 0..2 zamiast 1..3 w protokole

Jeżeli polecenie mówi, że klient wysyła liczbę z zakresu [1, 3], to kod powinien sprawdzać znaki '1'..'3', a nie '0'..'2'.

Listing 47: Poprawne mapowanie głosu

```
1 if (ch >= '1' && ch <= '3') {
2     int candidate_index = ch - '1'; /* 0..2 internally */
3     votes[elector_id] = candidate_index;
4 }
```

15.4 Błąd: wysyłanie sizeof(MSG) zamiast strlen(MSG)

Listing 48: Bezpieczniejszy zapis tekstu

```
1 #define WELCOME_MSG "Welcome, elector!\n"
2
3 bulk_write(fd, WELCOME_MSG, strlen(WELCOME_MSG));
```

sizeof(WELCOME_MSG) obejmuje znak końca napisu '\0'. Zwykle nie chcesz wysłać go przez TCP.

15.5 Błąd: brak czyszczenia stanu przy rozłączeniu

Jeżeli elektor 1 się rozłączył, ale w tablicy dalej ma connected = 1, to nie połączy się ponownie. Przy rozłączeniu trzeba zwolnić jego miejsce.

Listing 49: Czyszczenie zajętości elektora przy rozłączeniu

```
1 if (client->phase == IDENTIFIED) {
2     elector_fd[client->elector_id] = -1;
3 }
```

15.6 Błąd: pthread_create sprawdzany przez < 0

pthread_create zwraca 0 przy sukcesie albo dodatni kod błędu. Poprawny test:

Listing 50: Poprawne sprawdzenie pthread_create

```
1 int err = pthread_create(&tid, NULL, thread_func, arg);
2 if (err != 0) {
3     errno = err;
4     ERR("pthread_create");
5 }
```

16 Wątki i dane współdzielone

Jeżeli serwer ma dodatkowy wątek, np. wątek UDP wysyłający wyniki, pojawia się problem data race. Gdy jeden wątek zapisuje głosy, a drugi wątek je czyta, potrzebna jest synchronizacja.

Listing 51: Mutex do ochrony wyników

```
1 typedef struct election_state {
2     int votes[ELECTORS_NUM];
3     pthread_mutex_t mutex;
4 } election_state;
5
6 static void set_vote(election_state *st, int elector, int candidate)
7 {
8     pthread_mutex_lock(&st->mutex);
9     st->votes[elector] = candidate;
10    pthread_mutex_unlock(&st->mutex);
11 }
12
13 static void snapshot_votes(election_state *st, int out[ELECTORS_NUM])
14 {
15     pthread_mutex_lock(&st->mutex);
16     memcpy(out, st->votes, sizeof(st->votes));
17     pthread_mutex_unlock(&st->mutex);
18 }
```

Dobra zasada: mutex trzymaj krótko. Nie wykonuj długiego sendto, printf albo alokacji pamięci, jeśli nie musisz trzymać blokady. Lepiej zrobić kopię danych pod mutexem, odblokować, a potem wysłać.

17 Bezpieczne kończenie serwera

Po SIGINT serwer powinien:

1. przerwać pętlę epoll_wait,
2. zamknąć aktywne połączenia klientów,
3. zamknąć socket nasłuchujący,
4. zamknąć deskryptor epoll,
5. zatrzymać dodatkowe wątki,
6. zniszczyć mutexy,
7. wypisać końcowy wynik.

Minimalny wzorzec:

Listing 52: Sprzątanie zasobów po pętli serwera

```
1 while (do_work) {
2     int n = epoll_wait(efd, events, MAX_EVENTS, -1);
3     if (n < 0) {
```

```

4     if (errno == EINTR)
5         continue;
6     ERR("epoll_wait");
7 }
8 /* handle events */
9 }
10
11 for (int fd = 0; fd < FD_LIMIT; ++fd) {
12     if (clients[fd].active)
13         disconnect_client(efd, fd);
14 }
15
16 close(efd);
17 close(sfd);

```

Jeżeli `epoll_wait` jest przerwane przez sygnał, zwróci -1 z `errno == EINTR`. Wtedy sprawdzasz flagę `do_work`. Jeżeli handler ustawił `do_work = 0`, pętla skończy się.

18 Jak policzyć wynik wyborów

Wygodny format wewnętrzny:

- `votes[elector] == -1` - elektor jeszcze nie głosował,
- `votes[elector] == 0` - kandydat 1,
- `votes[elector] == 1` - kandydat 2,
- `votes[elector] == 2` - kandydat 3.

Liczenie:

Listing 53: Liczenie głosów dla kandydatów

```

1 #define CANDIDATES_NUM 3
2
3 static void print_results(int votes[ELECTORS_NUM])
4 {
5     int count[CANDIDATES_NUM] = {0};
6
7     for (int e = 0; e < ELECTORS_NUM; ++e) {
8         if (votes[e] >= 0 && votes[e] < CANDIDATES_NUM)
9             count[votes[e]]++;
10    }
11
12    for (int c = 0; c < CANDIDATES_NUM; ++c)
13        printf("Candidate %d: %d votes\n", c + 1, count[c]);
14 }

```

19 Kompilacja

Typowa komenda:

Listing 54: Kompilacja programu serwera

```
gcc -Wall -Wextra -Werror -std=c17 -pthread sop-hre.c -o sop-hre
```

Jeżeli używasz `EPOLLRDHUP`, upewnij się, że przed nagłówkami jest zdefiniowane `_GNU_SOURCE`. W `l4-common.h` już jest:

Listing 55: `_GNU_SOURCE` w `l4-common.h`

```
#define _GNU_SOURCE
```

Jeśli jednak w innym projekcie włączasz nagłówki systemowe przed `l4-common.h`, najlepiej dodać `-D_GNU_SOURCE` do kompilacji:

Listing 56: Kompilacja z *GNU_SOURCE*

```
1 gcc -D_GNU_SOURCE -Wall -Wextra -Werror -std=c17 -pthread server.c -o server
```

20 Krótka ściągą: co robić w którym miejscu

Miejsce w kodzie	Co robić
Start programu	Parsuj port, ustaw handler SIGINT, zainicjalizuj tablice klientów i głosów.
Tworzenie serwera	<code>bind_tcp_socket(port, backlog)</code> , potem opcjonalnie <code>set_nonblock(sfd)</code> .
Start epoll	<code>epoll_create1(0)</code> , dodaj <code>sfd</code> z <code>EPOLLIN</code> .
Zdarzenie na <code>sfd</code>	Wykonuj <code>accept()</code> / <code>add_new_client()</code> i dodaj każdego klienta do <code>epoll</code> .
Zdarzenie na kliencie	Sprawdź <code>EPOLLERR/HUP/RDHUP</code> , potem czytaj <code>read()</code> .
<code>read() > 0</code>	Przetwarzaj odebrane bajty zgodnie ze stanem klienta.
<code>read() == 0</code>	Klient zamknął połączenie, wyczyść stan i zamknij <code>fd</code> .
<code>read() < 0</code>	<code>EAGAIN</code> ignoruj, inne błędy zwykle kończą połączenie klienta.
Identyfikacja	Przed identyfikacją akceptuj tylko <code>'1'..'7'</code> .
Głosowanie	Po identyfikacji akceptuj <code>'1'..'3'</code> , inne znaki ignoruj.
Koniec programu	Zamknij klientów, <code>efd</code> , <code>sfd</code> , wątki, mutexy; wypisz wynik.

21 Checklist przed oddaniem

- Czy serwer przyjmuje port z argumentu programu?
- Czy socket nasłuchujący jest dodany do `epoll`?
- Czy nowi klienci są dodawani do `epoll`?
- Czy jeden milczący klient nie blokuje innych?
- Czy `read() == 0` zamyka klienta, ale nie kończy serwera?
- Czy po rozłączeniu elektora może on połączyć się ponownie?
- Czy duplikat elektora jest odrzucany?
- Czy błędny identyfikator przed logowaniem zamyka połączenie?
- Czy błędne znaki po identyfikacji są ignorowane?
- Czy głosy `'1'..'3'` są mapowane na indeksy `0..2`?
- Czy przy wysyłaniu napisów używasz `strlen`, a nie przypadkowo `sizeof` z bajtem zero-wym?
- Czy przy nieblokujących socketach obsługujesz `EAGAIN/EWOULDBLOCK`?
- Czy zasoby są zamykane po `SIGINT`?
- Czy dane współdzielone z wątkiem są chronione mutexem?

22 Podsumowanie

Najważniejsza mentalna mapa serwera:

1. `bind_tcp_socket` tworzy socket nasłuchujący.
2. `epoll` pilnuje, kiedy coś można zrobić.
3. Zdarzenie na sockecie nasłuchującym oznacza `accept` nowego klienta.

4. Zdarzenie na sockecie klienta oznacza read, rozłączenie albo błąd.
5. Każdy klient powinien mieć swój stan.
6. TCP nie zna granic wiadomości, więc to protokół aplikacyjny musi je zdefiniować.
7. Rozłączenie klienta jest normalnym zdarzeniem, a nie powodem do zakończenia serwera.

Jeżeli dobrze rozumiesz te siedem punktów, większość zadań laboratoryjnych z TCP i epoll sprowadza się do dopisania konkretnej logiki protokołu: identyfikacji, głosowania, czatu, broadcastu albo liczenia wyników.